

Diagnosis: After detecting an anomaly, the next step for the system is to determine why it is occurring. This is called root cause analysis (RCA). Using AI/ML, historical data, and event correlation, the system can pinpoint the most likely source of the anomaly. For example, it may correlate a sudden database lag with a recent deployment, or a recently made change in configuration for the database.

Action: After determining the root cause, the system must determine the right action to remediate the issue. In practical terms, this could mean restarting a failing service, scaling up infrastructure, rolling back a bad deployment, or rerouting traffic. The hope in operational teams is that all this will be accomplished automatically and with little or no disruption to users.

Role of real-time data and feedback loops

Self-healing systems are only as intelligent as the data they have access to. For self-healing systems to function well, they require real-time visibility into all parts of the technology stack. Infrastructure health, application performance, user interaction data, and even things like external API failures or a cloud outage, are all components

that need to be monitored for self-healing to be viable.

More critically, feedback loops must be an intrinsic part of the process or capability. For instance, after every healing action, the system needs to validate the effects: was the service really amended? Were there any side effects? Did it identify the root cause correctly?

This iterative learning process allows the system to enhance its predictions and ultimately improve accuracy over time. For instance, if a certain memory leak occurs multiple times, the self-healing system can escalate the issue, or it can identify it and suggest a fix at the code level. It can even change its thresholds in case the issue comes up again.

Real-time data gives the system its eyes and ears, and feedback loops give it the brain and memory. This self-healing intelligence advances with each incident it responds to.

Organisations are applying AIOps with the intent of removing manual processes, lowering downtime, and increasing resiliency. Table 2 lists three real-world applications of AIOps.

Let's look at the available commercial and open source AIOps tools and how they enable observability-led intelligence.

Commercial AIOps platforms

Several commercial AIOps platforms provide end-to-end solutions with real-time monitoring, machine learning, and automated incident response. Here are four leading contenders.

Dynatrace: Dynatrace has a well-known AI engine called Davis. They sell full-stack observability and have their functionality integrated into one workflow automation system with auto-discovery, real-time root cause analysis, and anomaly detection. They support auto-remediation of issues (via workflow integrations into other tools like ServiceNow or AWS Lambda).

Splunk (ITSI): Splunk's IT Service Intelligence (ITSI) platform utilises machine learning to help detect patterns, correlate events, and prioritise incidents. Through deep log analytics and predictive insights, Splunk helps teams prevent outages and launch automated responses.

Moogsoft: Moogsoft is focused on noise reduction phenomena and event-correlation functionality. It clusters similar alerted items through its AI and determines the probable cause of alerts. Its collaboration function helps DevOps teams through a unified event pipeline inquiry.

Datadog: Datadog includes support for AIOps and can aggregate all AIOps

Table 2: Real-world applications of AIOps

Scenario	Traditional approach	AIOps-driven solution	Key benefits of AIOps
Auto-remediation of failed deployments	Manual monitoring of deployments. Engineers roll back when issues are detected post-failure (often based on user reports or alerts).	Real-time anomaly detection in logs and KPIs. Automatic roll-back to previous stable version. Incident ticket generated with context.	Reduces downtime; Speeds up recovery; Maintains user experience; Less manual intervention
Predictive scaling in cloud infrastructure	Reactive scaling after hitting resource limits. Often leads to latency, crashes, or overprovisioning.	Machine learning forecasts resource needs based on traffic trends and usage history. Resources are scaled proactively.	Prevents performance issues; Improves cost-efficiency; Enhances user experience; Supports business continuity
Alert noise reduction in SRE teams	SREs face alert floods; many are false positives or duplicates. Manual triage is required to find root causes.	Intelligent correlation and deduplication of alerts. One consolidated, high-priority alert with root cause and action suggestions.	Reduces alert fatigue; Improves response time; Enables focus on critical issues; Enhances team productivity